

SIMATS ENGINEERING

CO5- ASSESSMENT TOOL 3 – Technical Presentation

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 10%

Time Duration: 1 Hour

QUESTIONS: 5

Total Marks:50

Code of Conduct:

I __K.Abhishek dass(192212301)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

1. System Analysis Presentation

Present a reverse engineering analysis of an AI-based system, explaining its architecture, components, and working process using diagrams.

2. Architecture Reconstruction Demo

Prepare a technical presentation to reconstruct the architecture of a given software/AI system, highlighting module interactions and data flow.

3. Algorithm & Logic Identification

Present how you identified algorithms and decision-making logic in a reverse-engineered system, including methodology and tools used.

4. Data Flow & Processing Pipeline

Explain through a presentation the data flow, processing pipeline, and communication mechanisms in a reverse-engineered distributed system.

5. Enhancement & Redesign Proposal

Deliver a technical presentation proposing improvements or redesign strategies for an existing system based on reverse engineering findings.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

SJMATS ENGINEERING

Reverse-Engineering a Multi-Agent Traffic Route Optimization System

Technical Presentation • CO5 – Assessment Tool 3

Course: Artificial Intelligence (CSA1709)

Course Outcome: CO5 — Distributed AI systems using PySpark, RDDs & intelligent agents

Presented by: [Name / Register No.]

What This Presentation Covers

Five reverse-engineering deliverables mapped to the assessment brief

1

System Analysis

Architecture, components & working process of the traffic-agent system

2

Architecture Reconstruction

Module interactions and end-to-end data flow

3

Algorithm & Logic Identification

Methodology and tools used to uncover decision logic

4

Data Flow & Processing Pipeline

PySpark/RDD pipeline and inter-agent communication

5

Enhancement & Redesign Proposal

Improvements based on reverse-engineering findings

1. System Analysis — What the System Does

System Under Study

A distributed multi-agent traffic route optimization platform that ingests live GPS/IoT sensor feeds and computes least-congestion routes for a city road network in near real time.

Scale	10,000+ vehicle probes/min across city road graph
Paradigm	PySpark for distributed batch/stream processing
Intelligence	One negotiating agent per road zone
Goal	Minimize system-wide travel time, not just individual routes

Sensing Layer

GPS probes · IoT roadside sensors

Ingestion Layer

Kafka topics per road segment

Distributed Processing

Spark cluster · RDD transformations

Agent Layer

Zone agents negotiate routes

Actuation Layer

Route dispatch to driver apps

1. System Analysis — Core Components

Six components identified through reverse engineering, and the role each plays

1

Traffic Sensor Network

GPS + roadside IoT devices publishing position/speed events per road segment

2

Kafka Message Bus

Decouples sensors from processing; buffers bursts; one topic per zone

3

Spark Cluster (Driver + Executors)

Distributes map/reduce workloads for congestion scoring across executors

4

RDD Road-Graph Store

Immutable, partitioned representation of the road network for fast recomputation

5

Route-Planning Agents

One autonomous agent per zone computes candidate routes for its vehicles

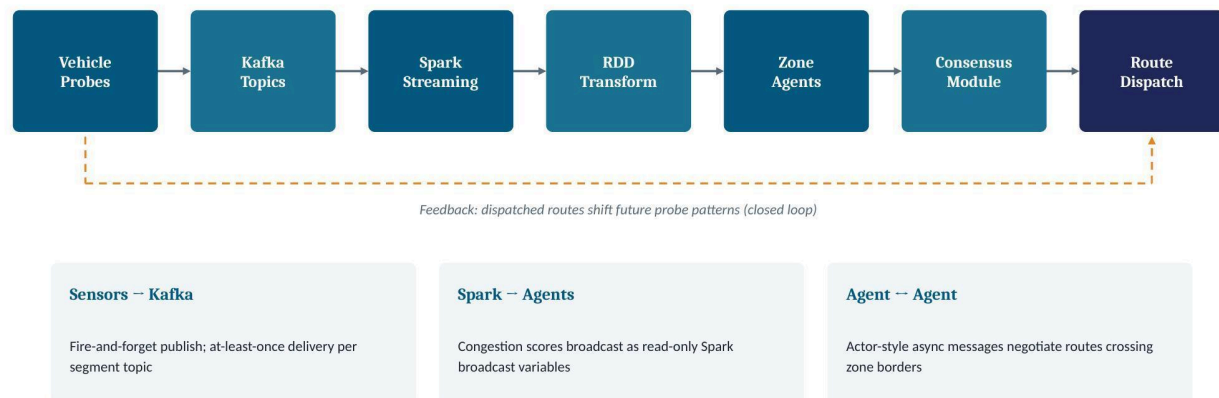
6

Coordination / Consensus Module

Resolves conflicting route proposals between neighboring agents

2. Architecture Reconstruction — Module Interactions

Reconstructed from log tracing, Spark job DAGs, and inter-agent message capture



3. Algorithm & Logic Identification

How the decision-making logic inside each module was uncovered

Algorithms Identified

- **Dijkstra / A* Search**

Shortest-path computation over the RDD-partitioned road graph

- **Reinforcement Learning Policy**

Per-zone agent predicts congestion and adapts routing weights over time

- **Auction-Based Negotiation**

Agents bid for shared road segments to resolve conflicting proposals

- **MapReduce Aggregation**

Reduce-by-key logic scores segment congestion from probe events

Methodology & Tools Used

- **Spark UI / History Server**

Inspected job DAGs and stage boundaries to trace transformation logic

- **Log & Message Tracing**

Captured inter-agent messages to infer negotiation rules

- **Static Code Reading**

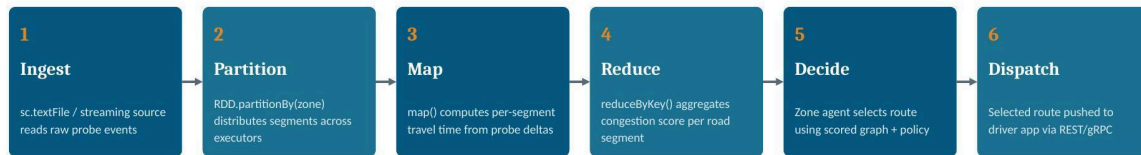
Reviewed agent decision functions and RDD lineage graphs

- **Behavioral / Black-box Testing**

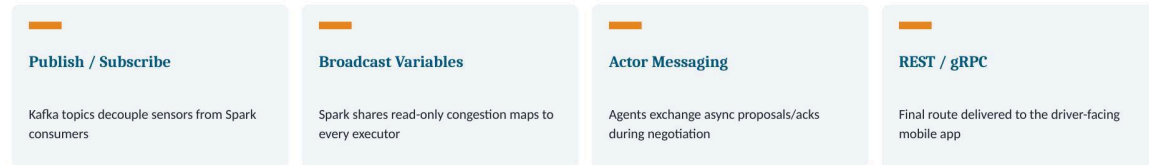
Fed synthetic traffic loads and observed routing responses

4. Data Flow & Processing Pipeline

PySpark/RDD pipeline turning raw probe events into a dispatched route



Communication mechanisms in play



5. Enhancement & Redesign Proposal

Limitations surfaced during reverse engineering, and the proposed redesign

Current Limitations

- No checkpointing — a driver failure restarts the whole streaming job
- Custom RDD graph operations reinvent what GraphX already offers
- All scoring runs in-cluster, adding latency for edge-adjacent zones
- Fixed executor count regardless of rush-hour vs. off-peak load

Proposed Redesign

- Add Spark checkpointing + write-ahead logs for fault-tolerant recovery
- Migrate road-graph operations to GraphX for optimized graph-parallel ops
- Push first-pass scoring to edge nodes near roadside sensors
- Enable dynamic executor auto-scaling tied to real-time probe volume

Key Takeaways

5

modules reconstructed
from the pipeline

4

algorithms identified
in agent logic

4

redesign proposals
for resilience & scale

-
- Reverse engineering revealed a layered, event-driven architecture rather than a monolithic planner
 - Distributed agents plus PySpark/RDDs give horizontal scale, but add coordination overhead
 - Targeted fixes — checkpointing, GraphX, edge scoring, auto-scaling — close the reliability gap

Thank you — questions welcome